
MODULE *pipeline_exec_ooo*

EXTENDS *Sequences, instructions_ooo, Integers, TLC, FiniteSets*
 VARIABLES *currCycle, prog, _IF, _ID, _RS, _FU, _COM, iMiss, FU, FULat*

$pipe_stages \triangleq \langle _IF, _ID, _RS, _FU, _COM \rangle$
 $Min(S) \triangleq \text{IF } S = \{\} \text{ THEN } 0 \text{ ELSE CHOOSE } x \in S : \forall y \in S : y \geq x$

$Init \triangleq \wedge currCycle = 0$
 $\wedge _IF = [i \in (1 \dots superscal) \mapsto [PC \mapsto empty, baseLat \mapsto 1, currLat \mapsto 1]]$
 $\wedge _ID = [i \in (1 \dots superscal) \mapsto [PC \mapsto empty]]$
 $\wedge _RS = [i \in (1 \dots N_FU) \mapsto \{\}]$
 $\wedge _FU = [i \in (1 \dots N_FU) \mapsto [PC \mapsto empty, baseLat \mapsto 1, currLat \mapsto 1, usage \mapsto 0]]$
 $\wedge _COM = [i \in (1 \dots superscal) \mapsto [PC \mapsto empty]]$

$NxtIFBusy \triangleq \exists i \in 1 \dots superscal : _IF[i].currLat < _IF[i].baseLat$ A I-cache miss stalls each pipeline
 $NxtFUBusy(i) \triangleq _FU[i].currLat < _FU[i].baseLat$

Set of (indexes of) instructions fully executed or about to finish execution
 $Exec \triangleq \{i \in 1 \dots Len(prog.exec) : \forall prog.exec[i].executed$
 $\quad \vee \exists j \in 1 \dots N_FU : prog.exec[i].PC = _FU[j].PC \wedge \neg NxtFUBusy(j)\}$

Set of (indexes of) instructions committed or about to leave the pipeline
 $Done \triangleq \{i \in 1 \dots Len(prog.exec) : \forall prog.exec[i].done$
 $\quad \vee \exists j \in 1 \dots superscal : prog.exec[i].PC = _COM[j].PC\}$

The decoded instructions to be executed in the i -th FU . There can be none or several per cycle.
 $FURouting(i) \triangleq \{_ID[j].PC : j \in \{k \in 1 \dots superscal : _ID[k].PC \neq empty \wedge FU[_ID[k].PC.pc] = i\}\}$

The next instruction to be executed in the i -th FU (not necessarily on the next cycle)
 $NxtFU(i) \triangleq \text{IF } NxtFUBusy(i)$
 THEN $empty$
 ELSE
 Older instruction (in program order) among the set of the waiting instructions in the corresponding
 RS or still in ID , whose all dependencies are satisfied (executed) or about to be.
 At most one instruction dispatched per cycle in each FU .
 LET $minReady \triangleq Min(\{x.pc : x \in$
 $\{y \in _RS[i] \cup FURouting(i) :$
 $\quad \forall z \in y.dep : z \in Exec\}$
 $\})$ IN
 IF $minReady = 0$ THEN $empty$
 ELSE CHOOSE $x \in _RS[i] \cup FURouting(i) : x.pc = minReady$

The set of (indexes of) the instructions that are:
 - $executed$ or about to finish execution
 - ready for in-order commit

- not already committed

$ReadyCOM(DoneCOM) \triangleq$ The set of (indexes of) instructions *potentially* ready for in-order commit (within the ROB), *i.e.*, whose all previous instructions (in program order) either have been committed/are about to leave the pipeline or are about to be committed in the same cycle (depending on $DoneCOM$).

LET $readyInOrder \triangleq \{j \in 1 \dots Len(prog.exec) : \forall k \in 1 \dots j-1 : k \in DoneCOM\}$

$(Exec \cap readyInOrder) \setminus DoneCOM$

Fetch (in-order)

$ProgressIF \triangleq$ LET $nxt(i) \triangleq next_instr(prog.rest, i)$ IN
 LET $cacheHit(i) \triangleq nxt(i) = empty \vee (nxt(i) \neq empty \wedge \neg iMiss[nxt(i).pc])$ IN
 LET $cacheMiss(i) \triangleq nxt(i) \neq empty \wedge iMiss[nxt(i).pc]$ IN
 LET $IFp(i) \triangleq$ IF $\neg NxtIFBusy \wedge cacheHit(i)$
 THEN $[PC \mapsto nxt(i), baseLat \mapsto 1, currLat \mapsto 1]$
 ELSE IF $\neg NxtIFBusy \wedge cacheMiss(i)$
 THEN $[PC \mapsto nxt(i), baseLat \mapsto missLat, currLat \mapsto 1]$
 ELSE $[_IF[i] \text{ EXCEPT } !.currLat = _IF[i].currLat + 1]$
 IN $_IF' = [i \in (1 \dots superscal) \mapsto IFp(i)]$

Decode (in-order)

$ProgressID \triangleq$ LET $nxtPc(i) \triangleq$ IF $\neg NxtIFBusy$ THEN $_IF[i].PC$ ELSE $empty$ IN
 $_ID' = [i \in (1 \dots superscal) \mapsto [PC \mapsto nxtPc(i)]]$

Issue

$ProgressRS \triangleq _RS' = [i \in (1 \dots N_FU) \mapsto (_RS[i] \cup FURouting(i))]$

Instructions are erased when beginning execution and do not go to RS s if executable at once:
 $\setminus \{NxtFU(i)\}$

OoO execution

$ProgressFU \triangleq$ LET $FUp(i) \triangleq$
 IF $\neg NxtFUBusy(i)$
 THEN
 IF $NxtFU(i) = empty$
 THEN $[PC \mapsto empty, baseLat \mapsto 1, currLat \mapsto 1, usage \mapsto _FU[i].usage]$
 ELSE $[PC \mapsto NxtFU(i), baseLat \mapsto FULat[NxtFU(i).pc],$
 $currLat \mapsto 1, usage \mapsto _FU[i].usage + 1]$
 ELSE $[_FU[i] \text{ EXCEPT } !.currLat = @ + 1, !.usage = @ + 1]$
 IN $_FU' = [i \in (1 \dots N_FU) \mapsto FUp(i)]$

Multiple in-order commit

RECURSIVE $nxtCOM(-)$
 $nxtCOM(s) \triangleq$ IF $s = 1$
 THEN
 The (index of the) instructions already ready for in-order commit

$ReadyCOM(Done)$
 ELSE
 The instructions ready for in-order commit, taking into account parallel commit (in the same cycle)
 $ReadyCOM(Done \cup \text{UNION } (\{nextCOM(j) : j \in 1 \dots s - 1\}))$

Superscalar commit: the oldest instruction (min. address) is selected among the ready instructions
 for each iteration of the multiple commit
 $ProgressCOM \triangleq \text{LET } nextPc(i) \triangleq$
 IF $nextCOM(i) \neq \{\}$ THEN $prog.exec[Min(nextCOM(i)).PC]$ ELSE $emptyIN$
 $_COM' = [i \in 1 \dots superscal \mapsto [PC \mapsto nextPc(i)]]$

RECURSIVE $execList(-)$
 $execList(i) \triangleq$ IF $i = 0$
 THEN $prog.exec$
 ELSE $exec_instr(execList(i - 1), [PC \mapsto next_instr(prog.rest, i), executed \mapsto \text{FALSE},$
 $done \mapsto \text{FALSE}, exStartTime \mapsto 0, comTime \mapsto 0])$

$prog.rest$ contains the instructions not already fetched
 $prog.exec$ acts as a (non-circular) ROB
 For simplification, instructions are stored while fetching and never removed.
 Entries indicate whether instructions are executed and whether done (pipeline left after commit)

$ProgressProg \triangleq \text{LET } restp \triangleq$ IF $\neg NxtIFBusy$ THEN $rest_instr(prog.rest, superscal)$ ELSE $prog.restIN$
 Add new instructions:
 LET $nwExec \triangleq$ IF $\neg NxtIFBusy$ THEN $execList(superscal)$ ELSE $prog.execIN$
 LET $execp \triangleq [i \in 1 \dots Len(nwExec) \mapsto$
 LET $instr \triangleq nwExec[i]IN$
 Update status:
 LET $executedp \triangleq i \in ExecIN$
 LET $donep \triangleq i \in DoneIN$
 LET $comTimep \triangleq$
 IF $\exists k \in (1 \dots superscal) : _COM[k] \neq empty \wedge _COM[k].PC = instr.PC$
 THEN $currCycle$
 ELSE $instr.comTimeIN$
 LET $exStartTimep \triangleq$
 IF $\exists k \in (1 \dots N_FU) : NxtFU(k) \neq empty \wedge NxtFU(k) = instr.PC$
 THEN $currCycle$
 ELSE $instr.exStartTimeIN$
 $[instr \text{ EXCEPT } !.executed = executedp, !.done = donep,$
 $!.exStartTime = exStartTimep, !.comTime = comTimep]$
]
 IN $prog' = [rest \mapsto restp, exec \mapsto execp]$

$Progress \triangleq ProgressIF \wedge ProgressID \wedge ProgressRS \wedge ProgressFU \wedge ProgressCOM \wedge ProgressProg$
 $TimeProgress \triangleq$ IF $\exists i \in 1 \dots Len(pipe_stages) :$

```

    LET  $x \triangleq \text{pipe\_stages}[i]$  IN  $x' \neq x$ 
    THEN  $\text{currCycle}' = \text{currCycle} + 1$   $\wedge \text{PrintT}(\text{Done})$ 
    ELSE UNCHANGED  $\langle \text{currCycle} \rangle \wedge \text{PrintT}(\text{currCycle})$ 

```

$\text{Next} \triangleq \text{Progress} \wedge \text{TimeProgress} \wedge \text{UNCHANGED} \langle i\text{Miss}, \text{FU}, \text{FULat} \rangle$

$\text{Spec} \triangleq \text{Init} \wedge \Box[\text{Next}]_{\langle \text{prog}, \text{pipe_stages}, \text{currCycle}, i\text{Miss}, \text{FU}, \text{FULat} \rangle}$

$\text{NotCompleted} \triangleq \text{Len}(\text{prog.rest}) > 0 \vee \text{Cardinality}(\text{Done}) < \text{Len}(\text{Program})$

$\text{IdealExec} \triangleq \text{currCycle} \leq \text{Len}(\text{pipe_stages}) + \text{Len}(\text{Program}) + 2$

$\text{TimeBound}(n) \triangleq \text{currCycle} \leq n$

$\text{InOrderComp} \triangleq \forall x \in \text{Done} : \forall i \in 1 \dots \text{Len}(\text{prog.exec}) : \text{prog.exec}[i].\text{PC.pc} < \text{prog.exec}[x].\text{PC.pc} \Rightarrow i \in \text{Done}$

$\text{DependExec} \triangleq \forall i \in 1 \dots N_{\text{FU}} : \text{FU}[i].\text{PC} \neq \text{empty} \Rightarrow \forall j \in \text{FU}[i].\text{PC.dep} : j \in \text{Exec}$

$\text{RAWDep} \triangleq \forall i \in 1 \dots \text{Len}(\text{Program}) : \forall d \in \text{Program}[i].\text{dep} : d < \text{Program}[i].\text{pc}$

* Modification History

* Last modified *Fri Jun 04 12:31:02 CEST 2021* by *benjaminbinder*

* Created *Thu Sep 10 13:44:58 CEST 2020* by *benjaminbinder*